



# Natural Language Processing

Sequence-to-sequence Models and Attention Mechanisms



# Syllabus of NLP course in University of Oslo, Norway

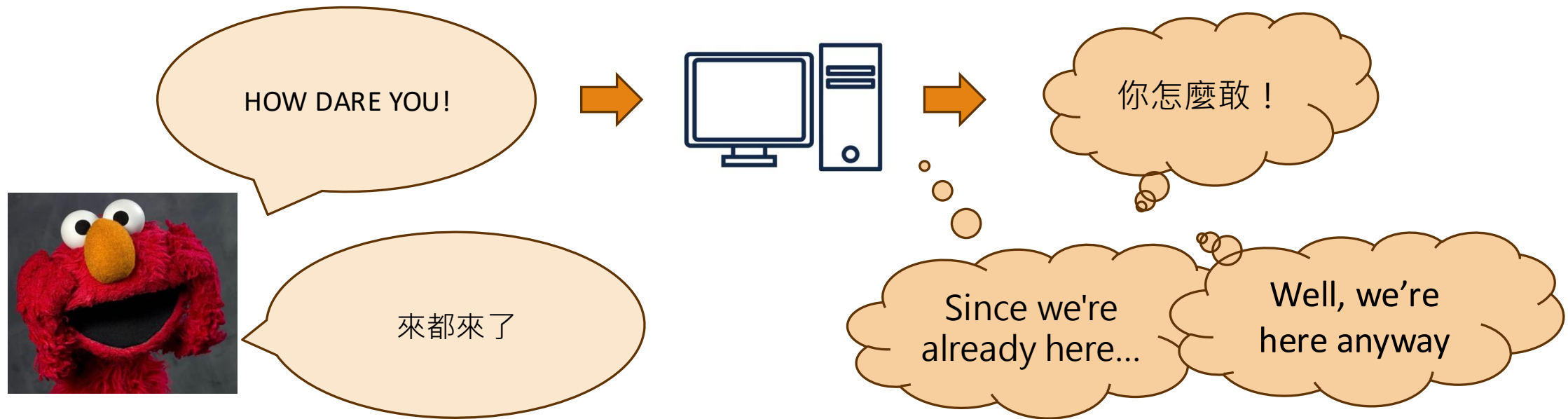
---

|        |                                               |                                                                        |
|--------|-----------------------------------------------|------------------------------------------------------------------------|
| Aug 18 | Introduction [YS+PL]                          |                                                                        |
| Aug 25 | Language models, tokenization [YS]            | <b>Oblig 1:</b> LMs and text classification<br><i>Deadline:</i> Sep 11 |
| Sep 1  | Text classification with log. regression [YS] |                                                                        |
| Sep 8  | Sequence labelling [YS]                       |                                                                        |
| Sep 15 | Neural networks [PL]                          | <b>Oblig 2:</b> Sequence labelling and NNs<br><i>Deadline:</i> Oct 2   |
| Sep 22 | Vector semantics, word embeddings [YS]        |                                                                        |
| Sep 29 | Transformers [YS]                             |                                                                        |
| Oct 6  | Large language models [PL]                    | <b>Oblig 3:</b> LLMs and RAG<br><i>Deadline:</i> Oct 23                |
| Oct 13 | Dialog systems I [PL]                         |                                                                        |
| Oct 20 | Dialog systems II, question answering [PL]    |                                                                        |
| Oct 28 | Machine translation [YS]                      | <b>Oblig 4:</b> dialogue systems and MT<br><i>Deadline:</i> Nov 13     |
| Nov 3  | Speech processing [PL]                        |                                                                        |
| Nov 10 | Ethics I [PL]                                 |                                                                        |
| Nov 17 | Ethics II, exam preparation [YS+PL]           | <b>Exam:</b> Dec 18                                                    |

# Machine Translation

Machine translation plays a pivotal role in the development of NLP.

- Many advancements in NLP language models were initially motivated by the need to address translation challenges.



# Challenges of Machine Translation

---

Unlike the other tasks like classification, the input and output of machine translation are **in different lengths**.



✗ We cannot just add an FFN at the end.

✓ The hidden state should be utilized to encode the original sequence and it should be passed to the generation process.

# Sequence to sequence Model

---

Sequence to Sequence (Seq2Seq) model

The core idea of the Seq2Seq model is to map **variable-length input sequences to variable-length output sequences**.

This flexibility enables its wide application across various tasks, such as:

- **Machine Translation**: Translating a sentence from one language to another.
- Text Summarization: Condensing long articles into shorter summaries.
- Dialogue Generation: Generating appropriate responses based on input contexts.

# Sequential Models

---

Sequence models, also known as time series models, are a class of neural network architectures designed to model sequences of data.

- **RNN**
- **LSTM**

Each output of a sequential model depends on the hidden state variable provided by the previous step.

# RNN for Sequence Generation

---

## 1. Input Encoding:

- The input sequence is fed into the RNN one element at a time.

## 2. Hidden State Update:

- The RNN updates its hidden state at each time step based on the current input element and the previous hidden state.

## 3. Output Generation:

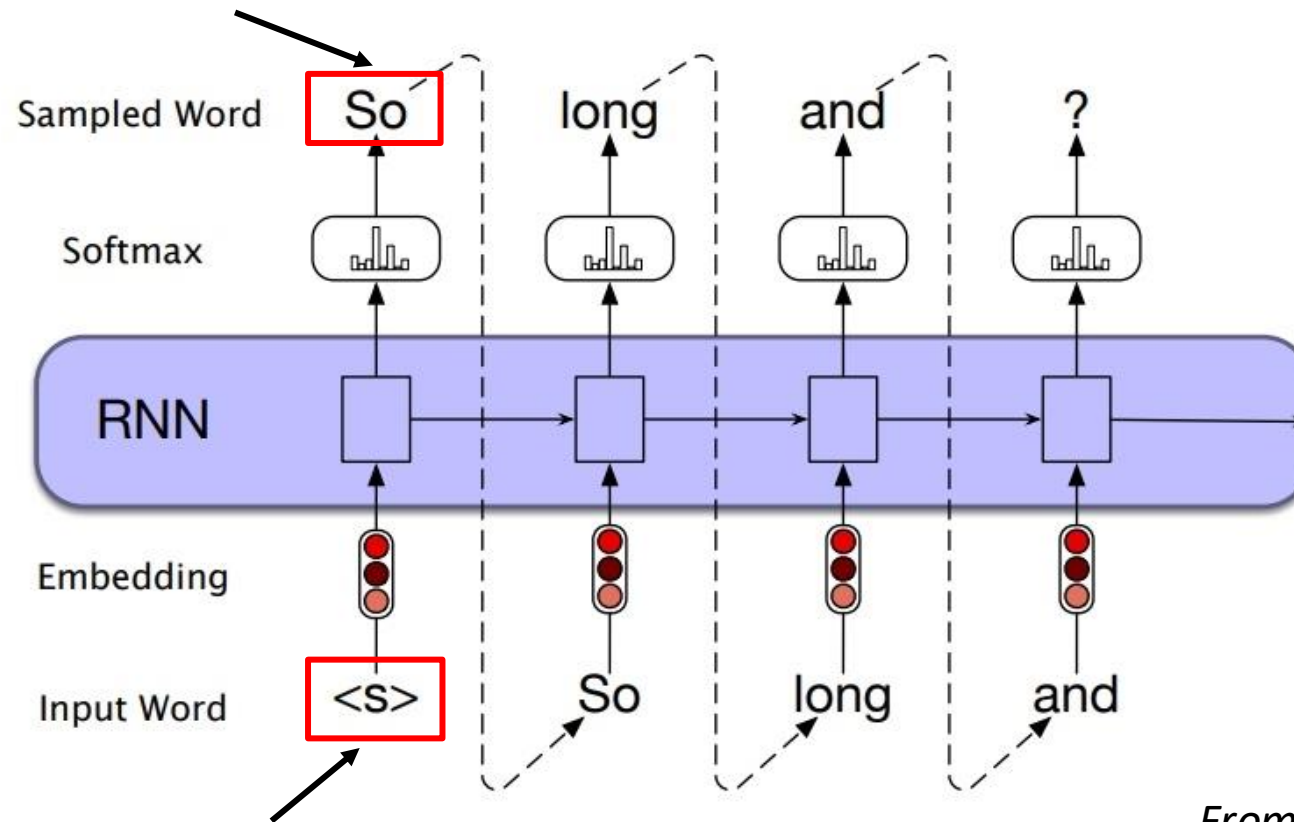
- Using the hidden states at each time step, the RNN generates the output sequence one element at a time.

## 4. Iteration:

- Steps 2-3 are iterated until the desired length of the output sequence is reached or until a specific termination condition is met (e.g., generating an end-of-sequence token).

# RNN for Sequence Generation

Predict the next token.

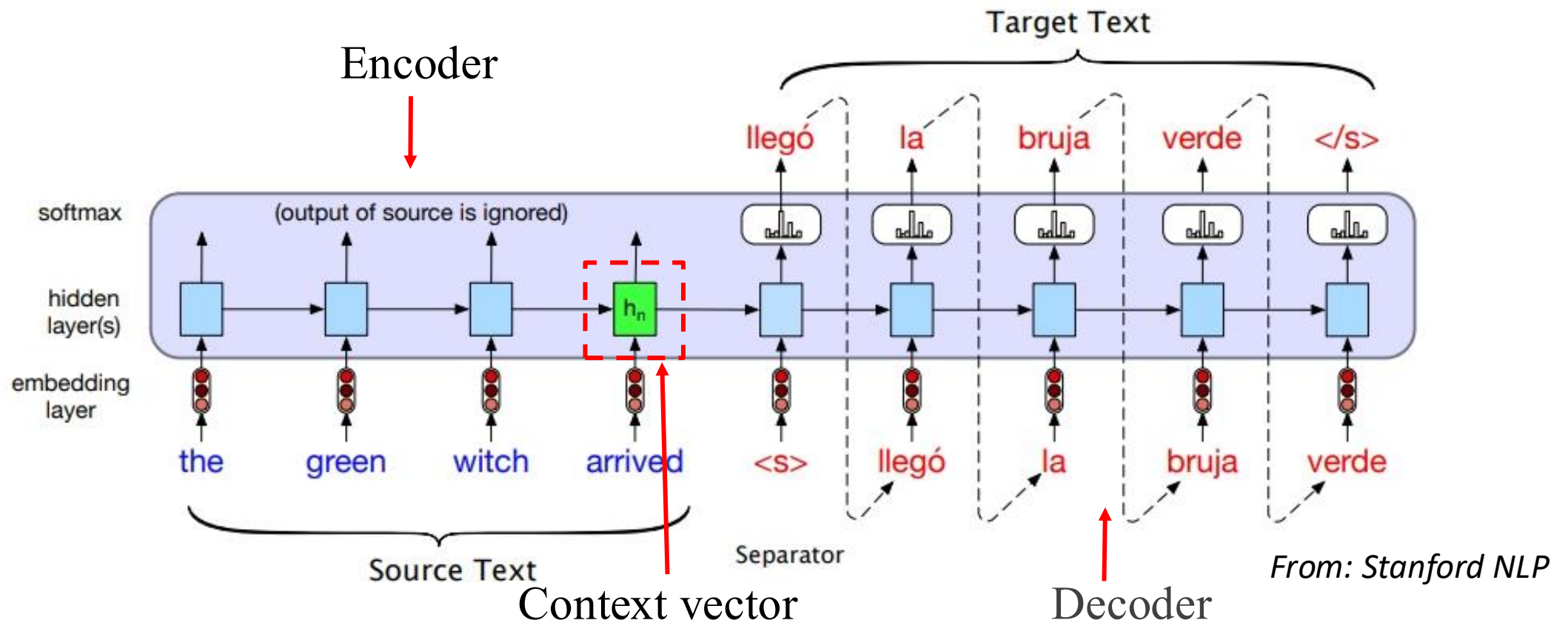


*From: Stanford NLP*

The sequence generation begin with a special token.

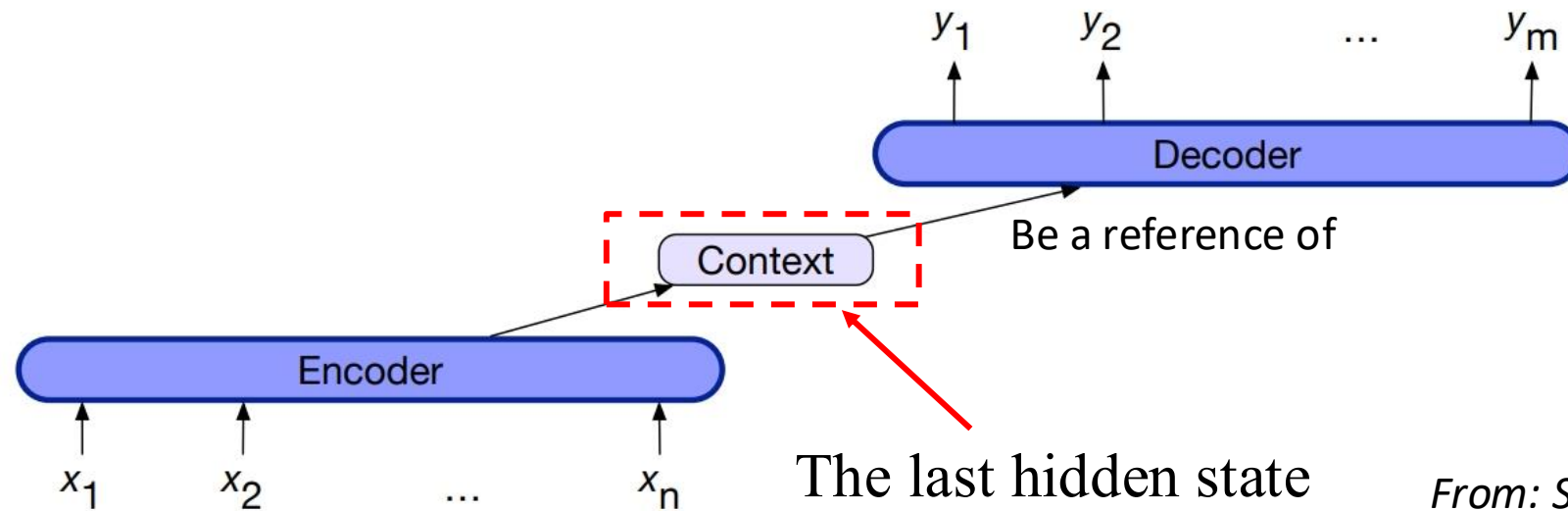
# RNN for Machine Translation

- Machine translation is a classic application of encoder-decoder architecture.



# RNN for Machine Translation

- The context vector is the last hidden state of the encoder and it contains all the information of the input.



*From: Stanford NLP*

# Encoder-decoder RNN

---

Encoder-decoder networks consist of three conceptual components:

- **An encoder** that accepts an input sequence,  $x_{1:n}$ , and generates a corresponding sequence of *contextualized representations*,  $h_{1:n}$ .
- **A context vector**, which is a function of  $h_{1:n}$ , and conveys the essence of the input to the decoder.
- **A decoder**, which accepts the context vector as input and generates an arbitrary length sequence of hidden states  $h_{1:m}$ , from which a corresponding sequence of output states  $y_{1:m}$ , can be obtained.

# Gradient Vanishing/Exploding Problem of RNN

---

- During backpropagation, gradients are computed with respect to the hidden states through the loss function at each time step.

$$y_t = g(Vh_t)$$

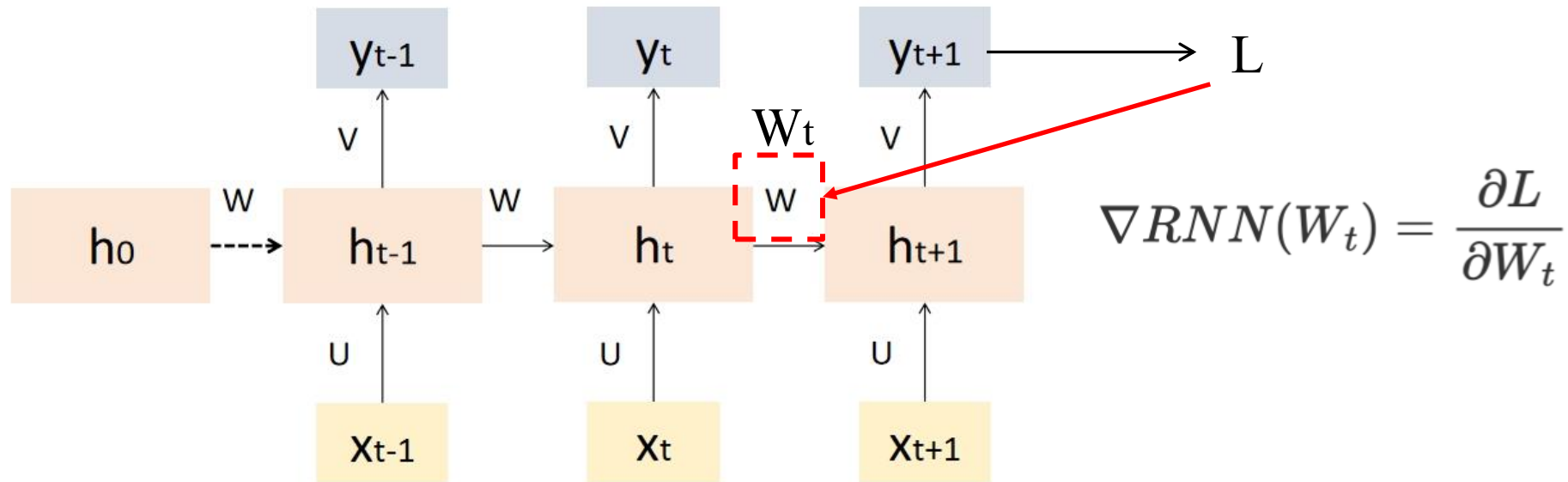
$$h_t = f(Ux_t + Wh_{t-1})$$

- Compute the gradient (use the parameter  $W$  as an example):

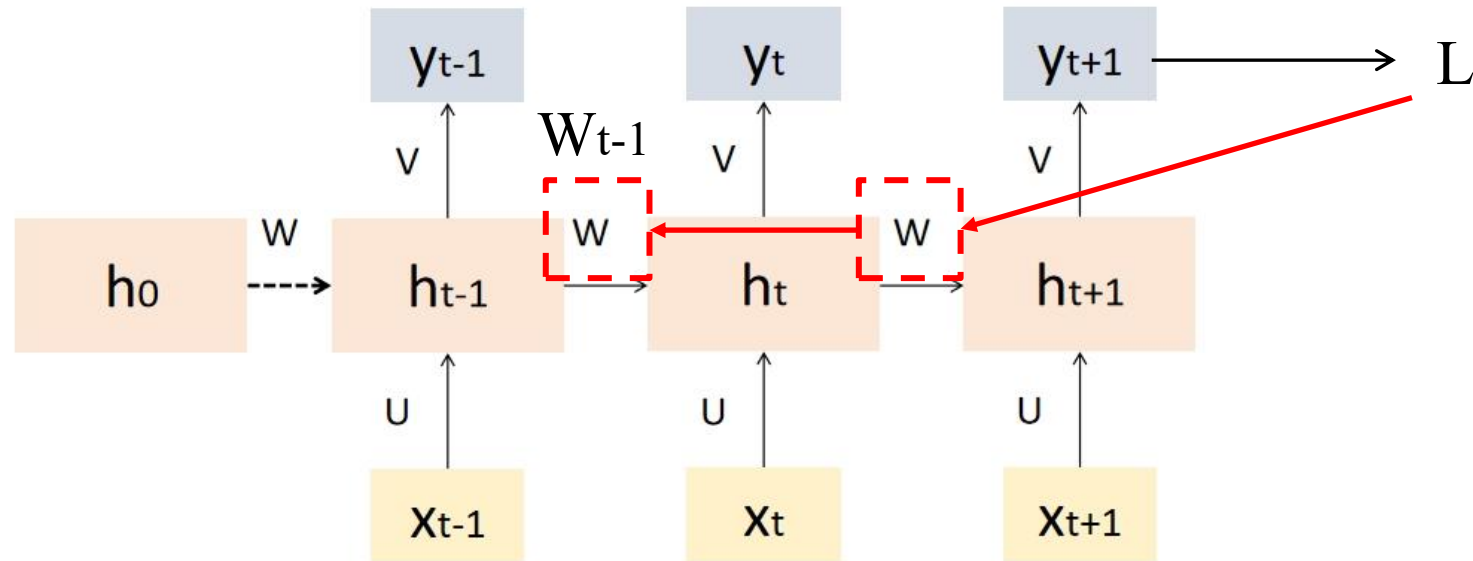
$$\frac{\partial L}{\partial W} = \sum_{t=1}^T \frac{\partial L}{\partial h_T} \cdot \boxed{\frac{\partial h_T}{\partial h_{T-1}}} \cdots \frac{\partial h_t}{\partial W}$$

The further back in time an input is, the more factors it needs to be multiplied by.

# Gradient Vanishing/Exploding Problem of RNN

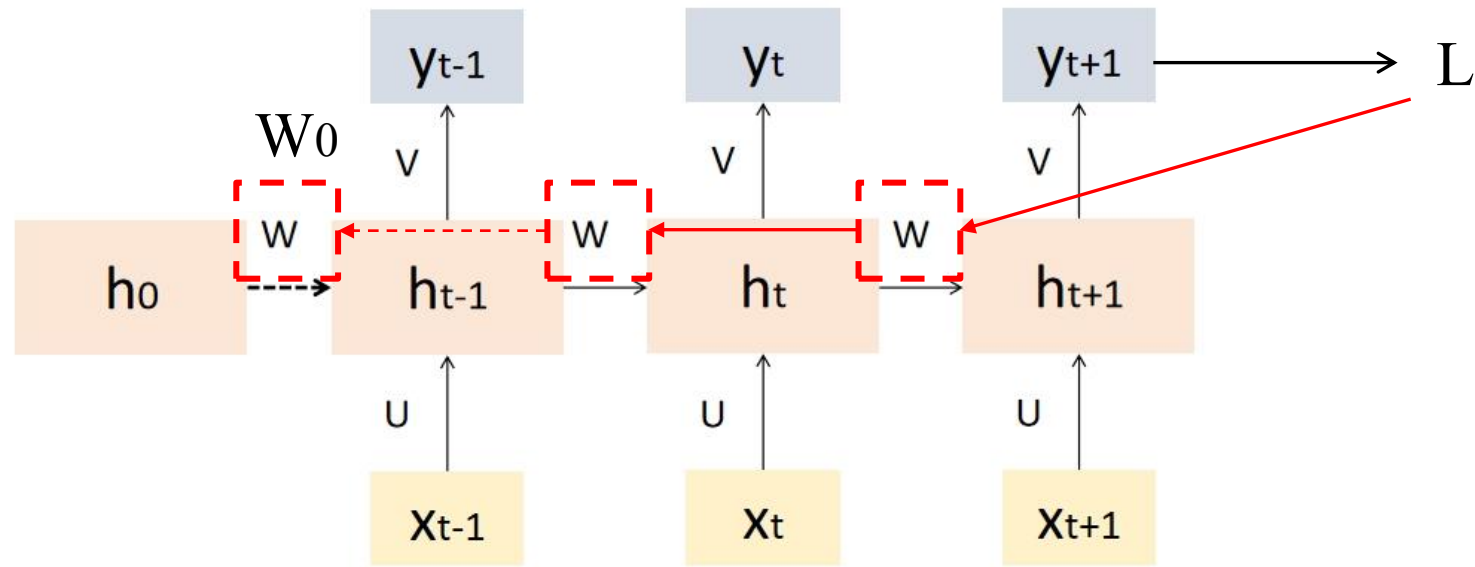


# Gradient Vanishing/Exploding Problem of RNN



$$\nabla RNN(W_{t-1}) = \frac{\partial L}{\partial W_{t-1}} = \frac{\partial L}{\partial W_t} \frac{\partial W_t}{\partial W_{t-1}}$$

# Gradient Vanishing/Exploding Problem of RNN

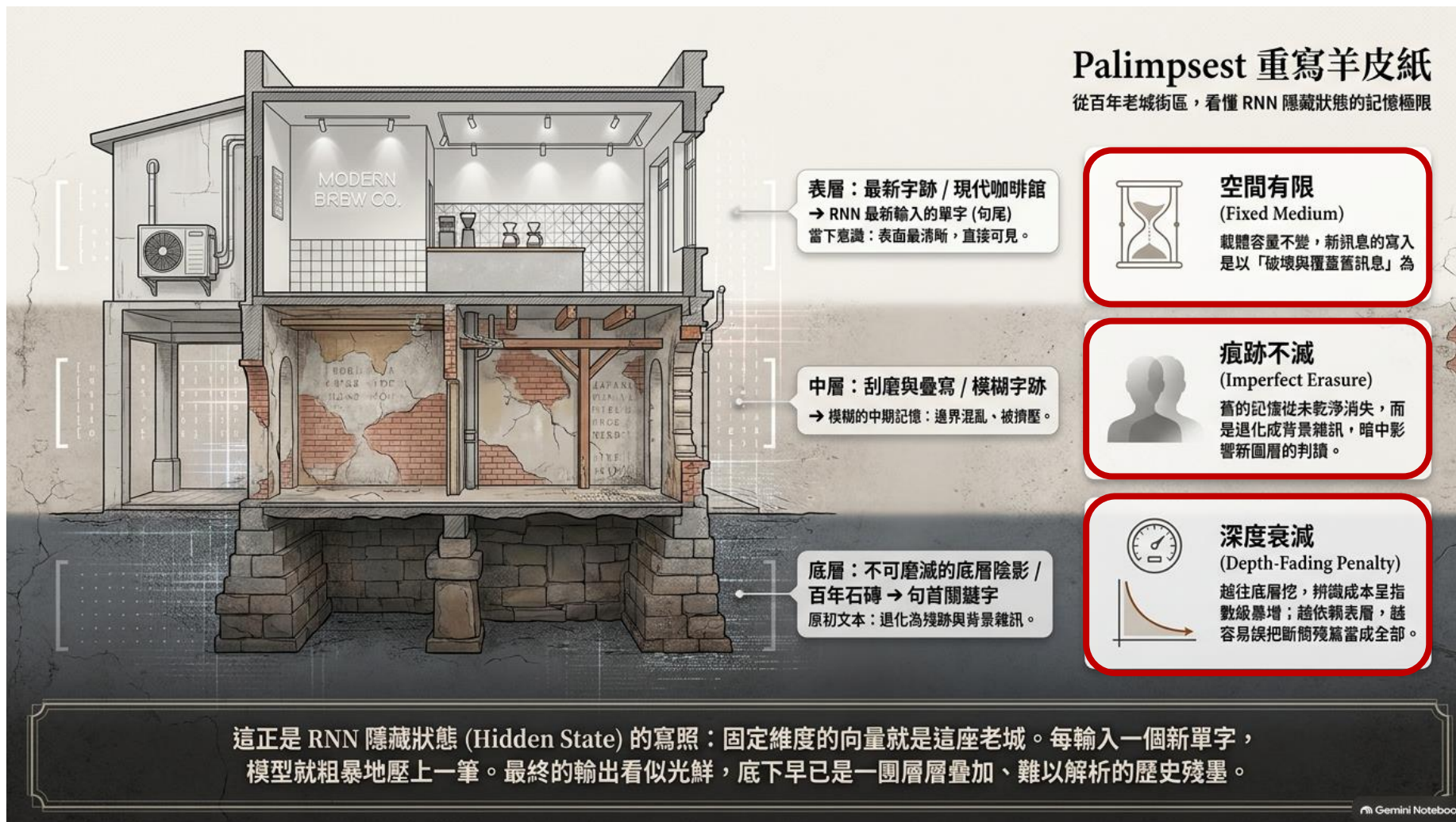


$$\nabla RNN(W_0) = \frac{\partial L}{\partial W_0} = \underbrace{\frac{\partial L}{\partial W_t} \frac{\partial W_t}{\partial W_{t-1}} \frac{\partial W_{t-1}}{\partial W_{t-2}} \cdots \frac{\partial W_1}{\partial W_0}}_{\text{gradient terms}}$$

If these terms  $< 1$ , the gradient decays exponentially (gradient vanishing)

If these terms  $> 1$ , the gradient grows rapidly (gradient exploding)

# Palimpsest (重寫羊皮紙)



# Gradient Vanishing Problem of RNN

---

Gradient vanishing results in :

- **Information Loss:**

As the number of time steps increases, gradient vanishing prevents RNNs from retaining important information from earlier time steps.

- **Training Instability:**

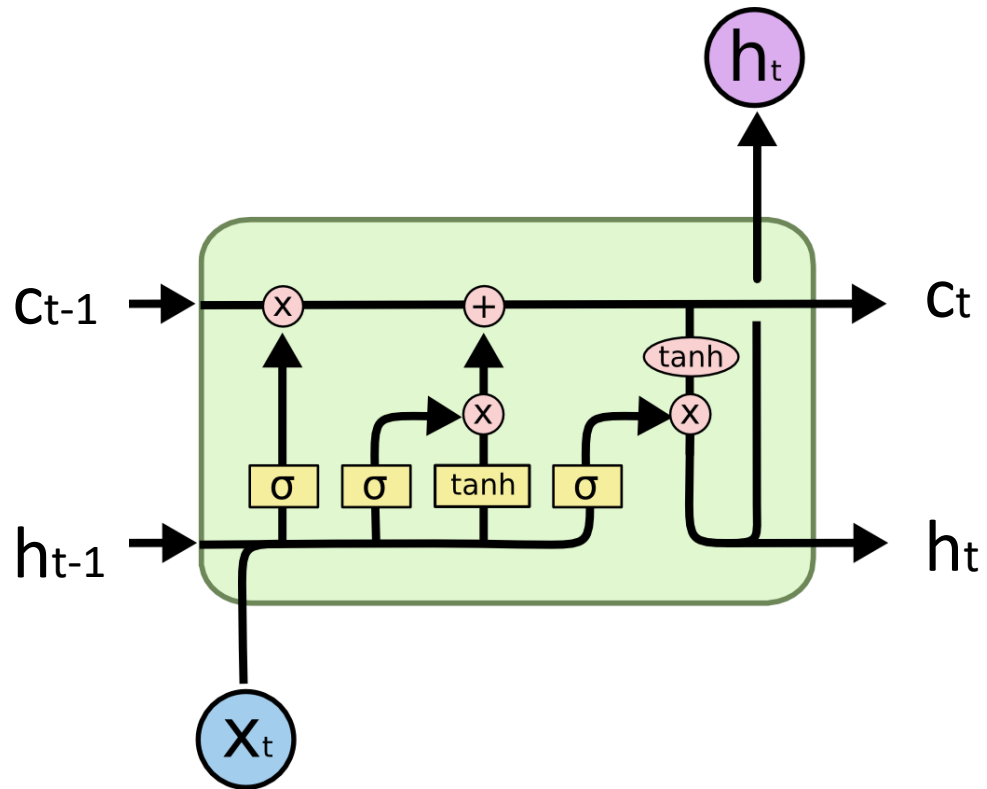
With the disappearance of gradients in early time steps, the update of network parameters becomes slow or even stagnates.

- **Difficulty in Generating Long Sequences:**

RNNs perform poorly in generating long sequences. Generated sequences may become incoherent or meaningless.

# Long Short-Term Memory

Long Short-Term Memory (LSTM) is proposed to solve Gradient Vanishing problem of RNNs.



$$R_t = \sigma_r(W_r x_t + U_r h_{t-1} + b_r) \quad \text{Forget gate}$$

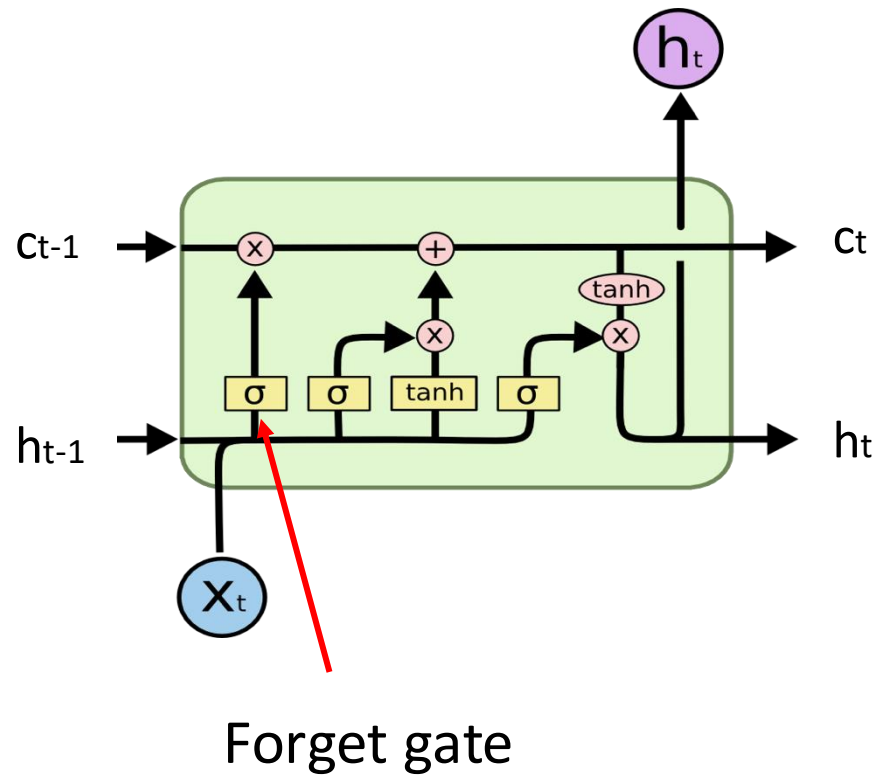
$$K_t = \sigma_r(W_k x_t + U_k h_{t-1} + b_k) \quad \text{Input gate}$$

$$V_t = \sigma_r(W_v x_t + U_v h_{t-1} + b_v) \quad \text{Output gate}$$

$$c_t = R_t \cdot c_{t-1} + K_t \cdot \sigma_c(W_c x_t + U_c h_{t-1} + b_c)$$

$$h_t = V_t \sigma_h(c_t)$$

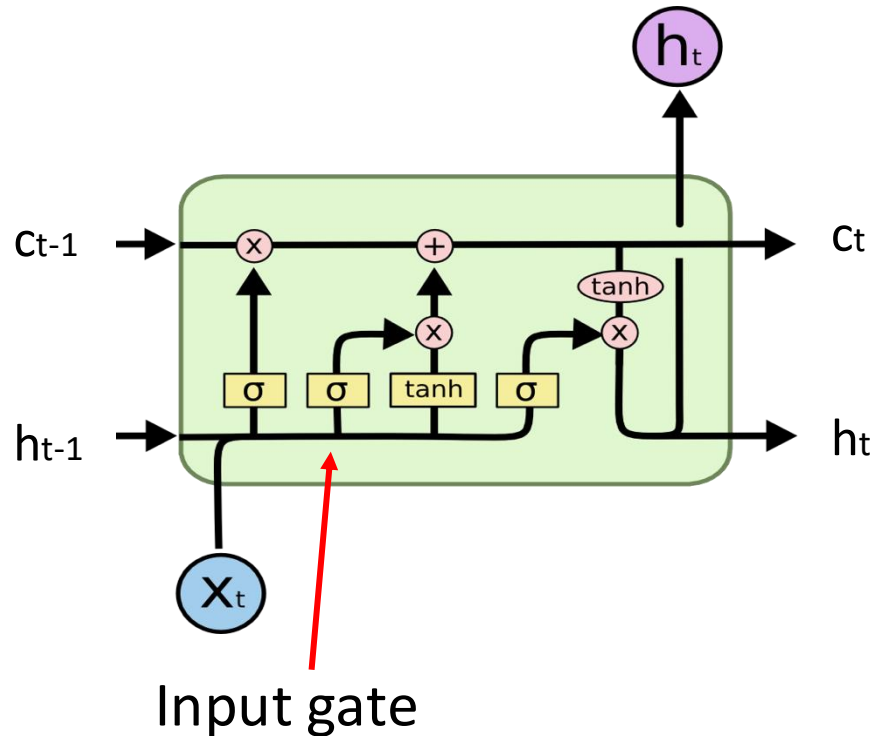
# Long Short-Term Memory



$$R_t = \sigma_r(W_r x_t + U_r h_{t-1} + b_r)$$

- **Forget Gate:** It uses a sigmoid function to determine which information from past memory should be forgotten or discarded.

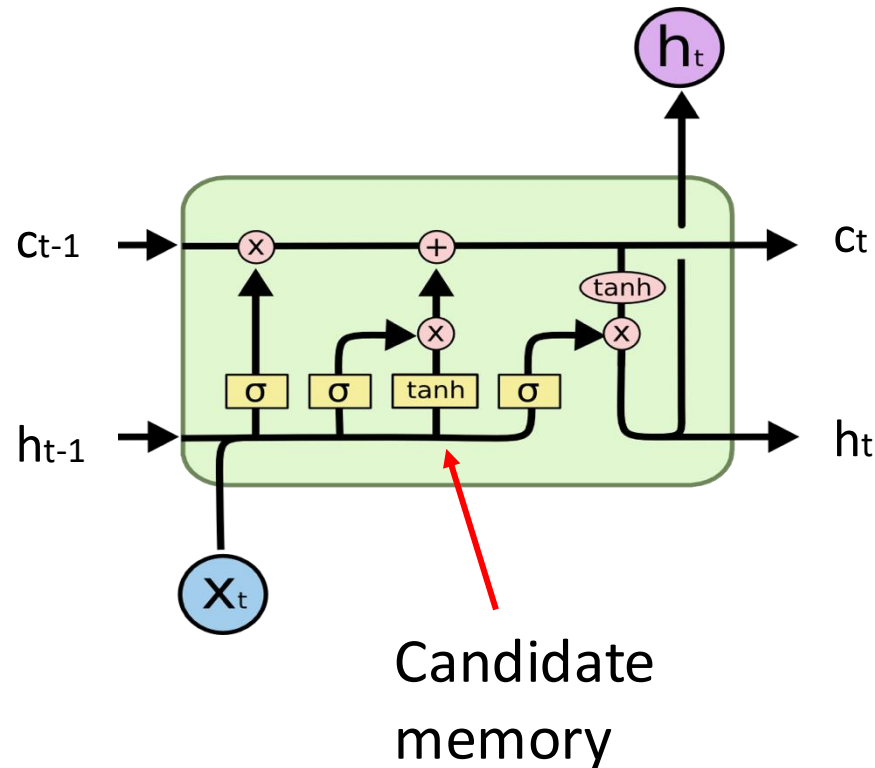
# Long Short-Term Memory



$$K_t = \sigma_r(W_k x_t + U_k h_{t-1} + b_k)$$

- **Input Gate:** It uses a sigmoid function to determine which information should be added to the cell state.

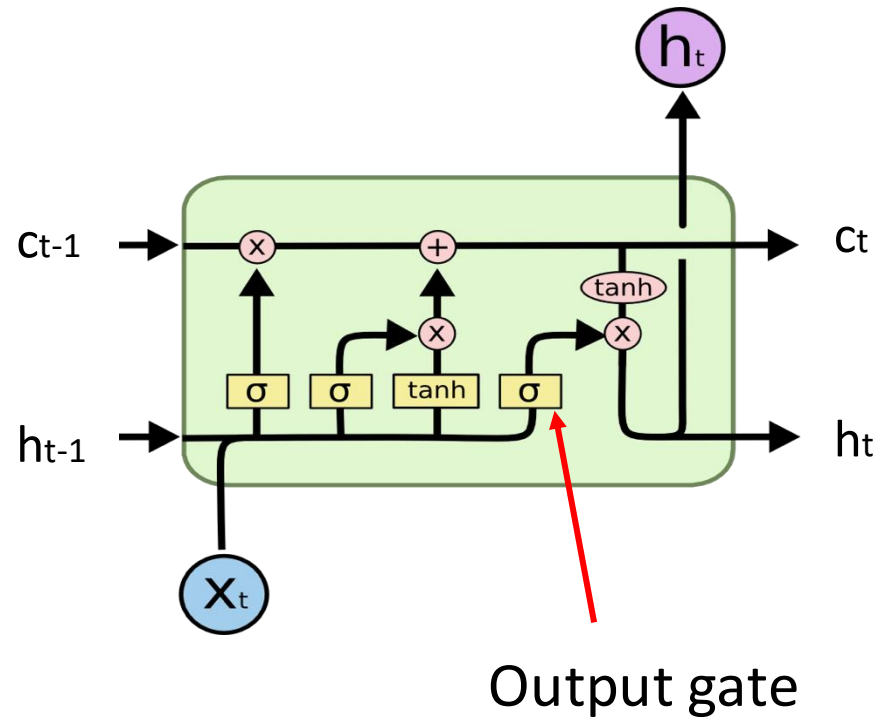
# Long Short-Term Memory



$$V_t = \sigma_r(W_v x_t + U_v h_{t-1} + b_v)$$

- **Candidate Memory:** The candidate memory represents potential **new** information that could be added to the cell state. It is generated by a tanh which is relatively closer to binary than sigmoid.

# Long Short-Term Memory



$$c_t = R_t \cdot c_{t-1} + K_t \cdot \sigma_c(W_c x_t + U_c h_{t-1} + b_c)$$

$$h_t = V_t \sigma_h(c_t)$$

- **Output Gate:** The output gate controls the flow of information from the cell state to the hidden state.

# How does LSTM work?

## Summary in plain words:

- **Forget gate** → drop “chasing” details.
- **Input gate** → add “climbing” action.
- **Candidate memory** → encode “climb + object.”
- **Output gate** → release “tree” at the right time.

*“The cat chased the mouse, and then it climbed a tree.”*



LSTM gate interpretations (language perspective)

## Forget Gate

- When the model reaches “climbed a tree,” it **reduces the importance of older context like “chased the mouse”** because the focus shifts to a new action.
- **Meaning:** drop past context that is less relevant for the current prediction.

## Input Gate

- At “climbed,” the model needs to **store this new action (climb) into the memory** since it defines what happens next.
- **Meaning:** decide which new information is worth adding to memory.

## Candidate Memory

- Here, the candidate representation encodes the semantics of “climb + tree.”
- **Meaning:** generate potential new content that could be added to the cell state.

## Output Gate

- When producing the word “tree,” the model **selects from memory the relevant semantic content (the location being climbed)** to expose as output.
- **Meaning:** choose which part of the memory should influence the current word generation.

# Long Short-Term Memory

---

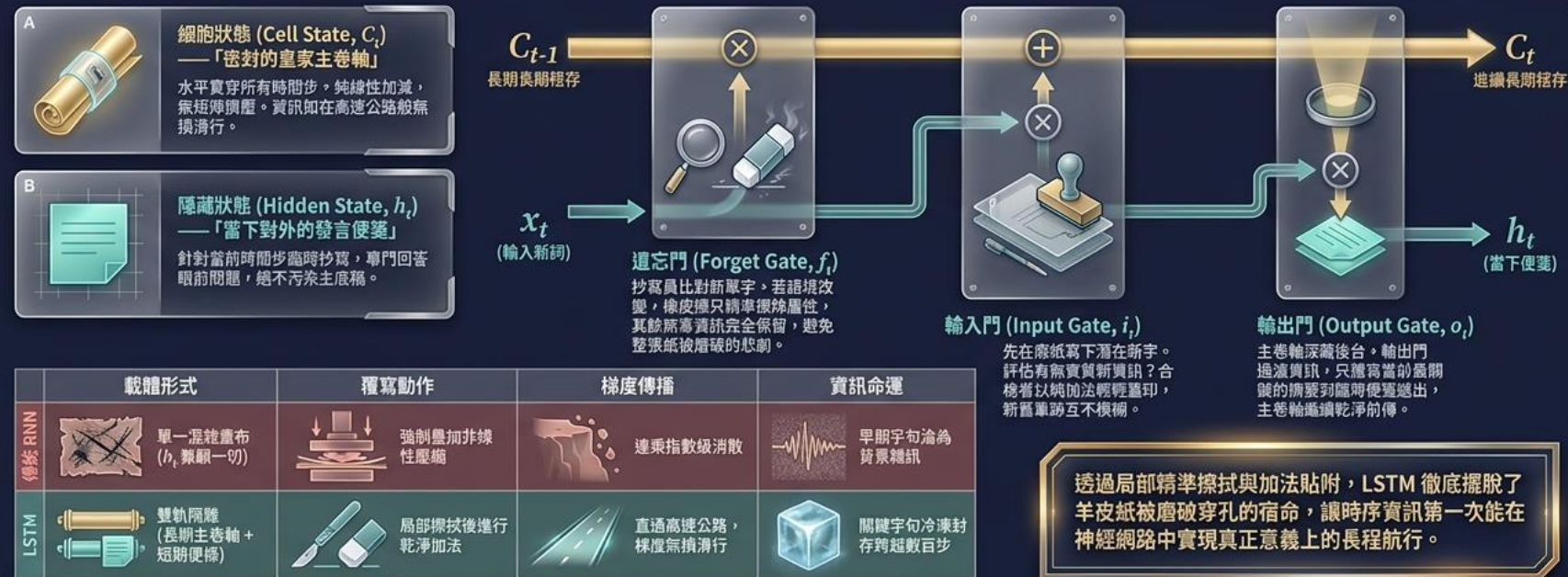
LSTM can partially avoid the vanishing gradient problem due to :

- **gating mechanisms & memory cells.**
  - These components enable LSTM to **selectively retain or discard information** over time.
  - It maintains stable gradient flow during training and capture long-term dependencies more effectively compared to traditional RNNs.

# Palimpsest (重寫羊皮紙) 2.0

## 從「刮削廢墟」到「精密輸送帶」： LSTM神經網路的檔案櫃管理法

傳統 RNN 像是在同一張固定大小的羊皮紙上粗暴刮除與重複書寫；LSTM 則為  
時序資訊配備了「雙軌隔離」與「三位把關者」，實現真正意義上的長程航行。



# Problems of time-series

---

Traditional sequential models, such as simple RNNs, suffer from the following issues:

- **Vanishing/Exploding Gradients:** although LSTM alleviate this problem, it is still existed.
- **Difficulty in Parallelization:** Because sequential models **rely on the previous time step's hidden state**, they are challenging to parallelize effectively, limiting their training efficiency on large-scale datasets and the size-growth of language models.

# Attention Mechanisms

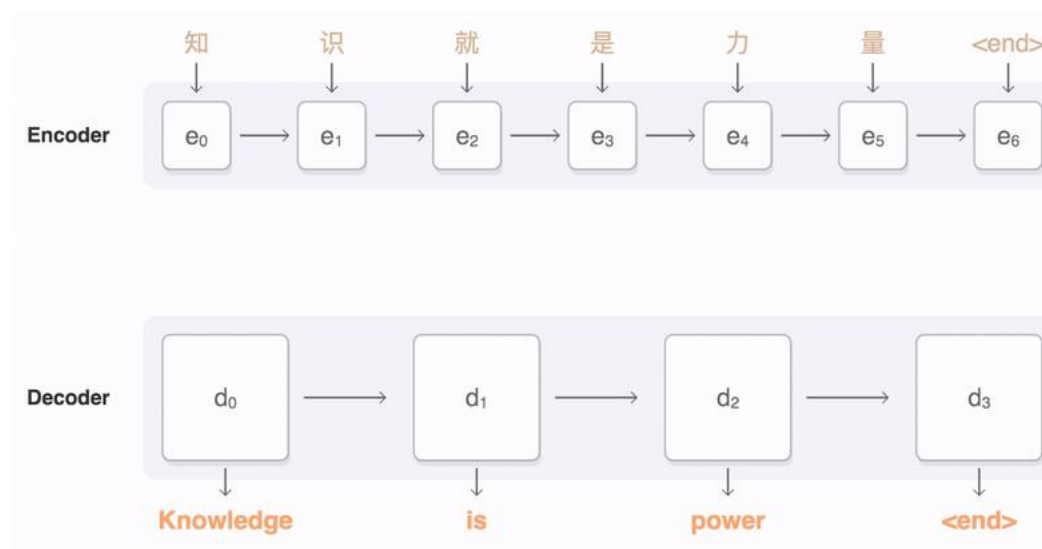
The attention mechanism solve the gradient problems and provided a parallelizable solution of LMs.

Core Idea: To enable a model to **focus on the most relevant parts** of the input sequence when making predictions or generating outputs.

- **Attention**

Attention with RNNs

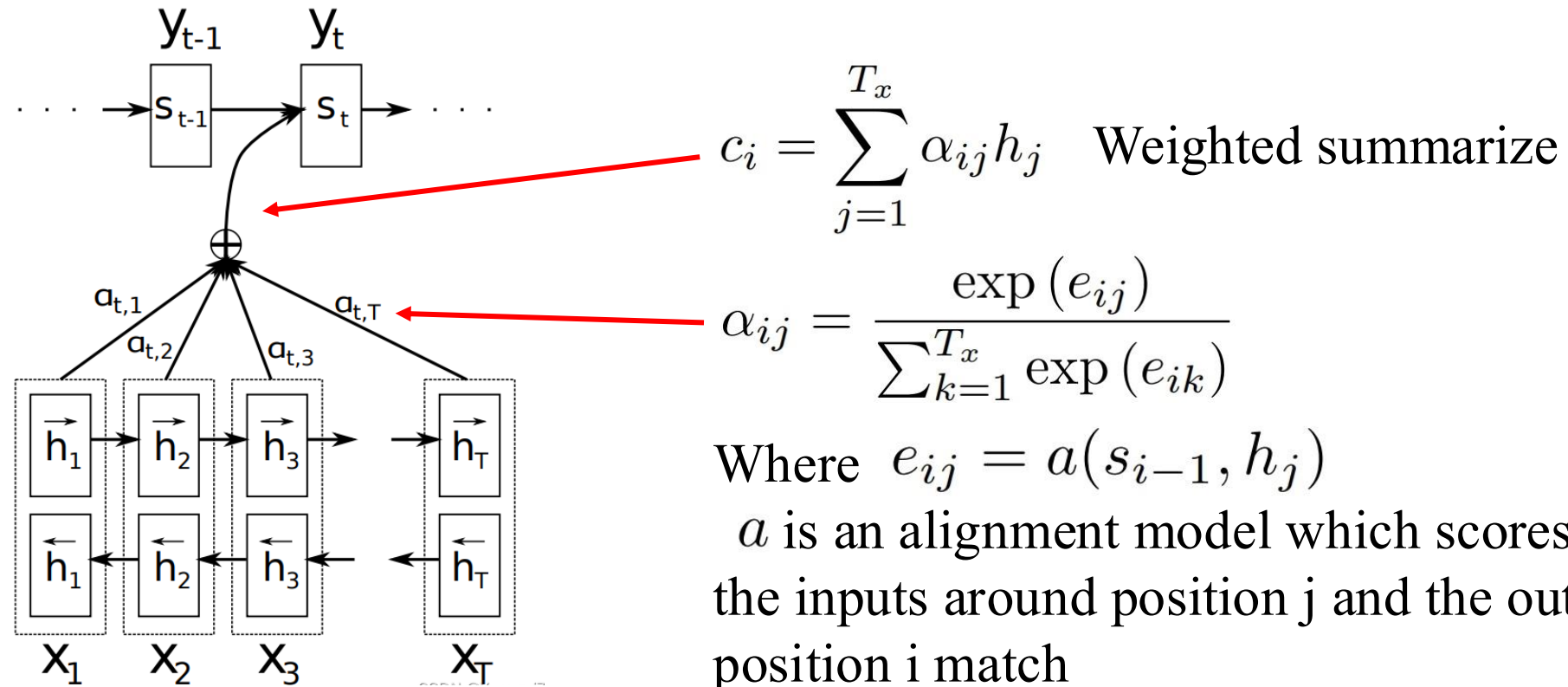
Attention without RNNs



From Google Seq2seq

# Attention

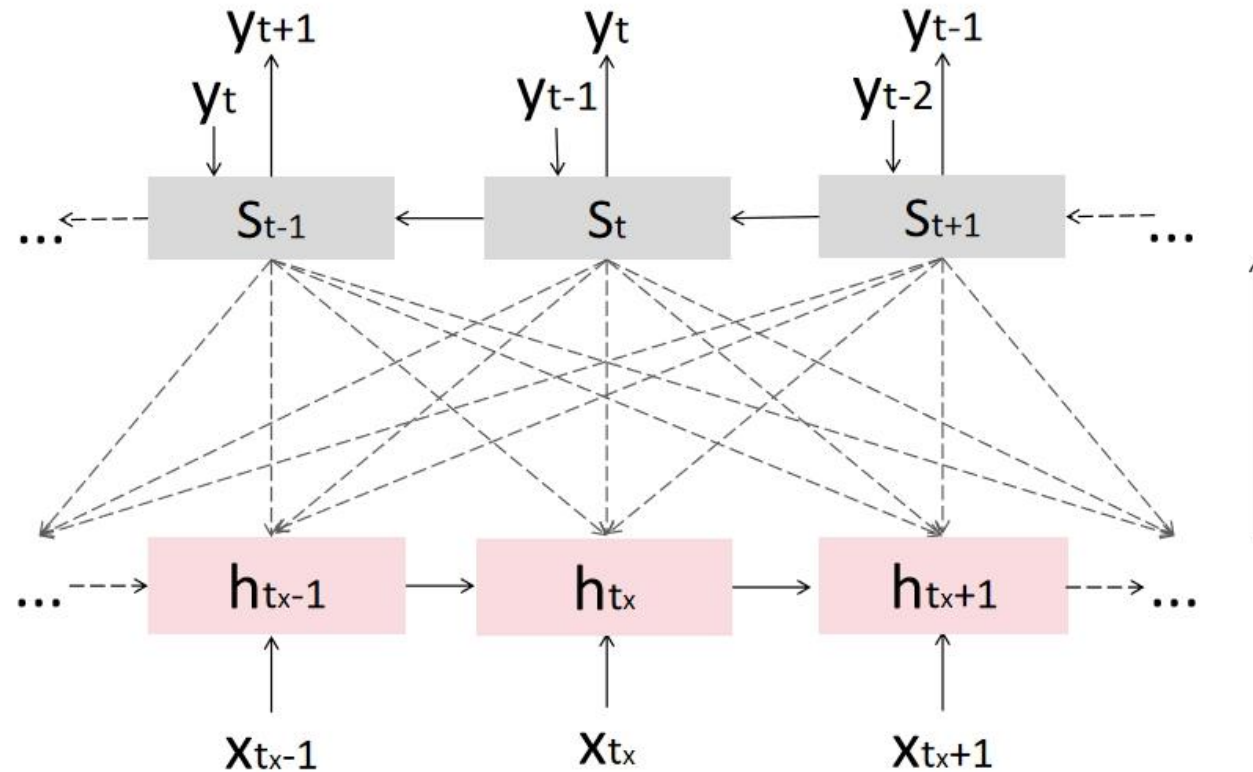
At the beginning the attention is associated with RNNs.



From: Attention 2015

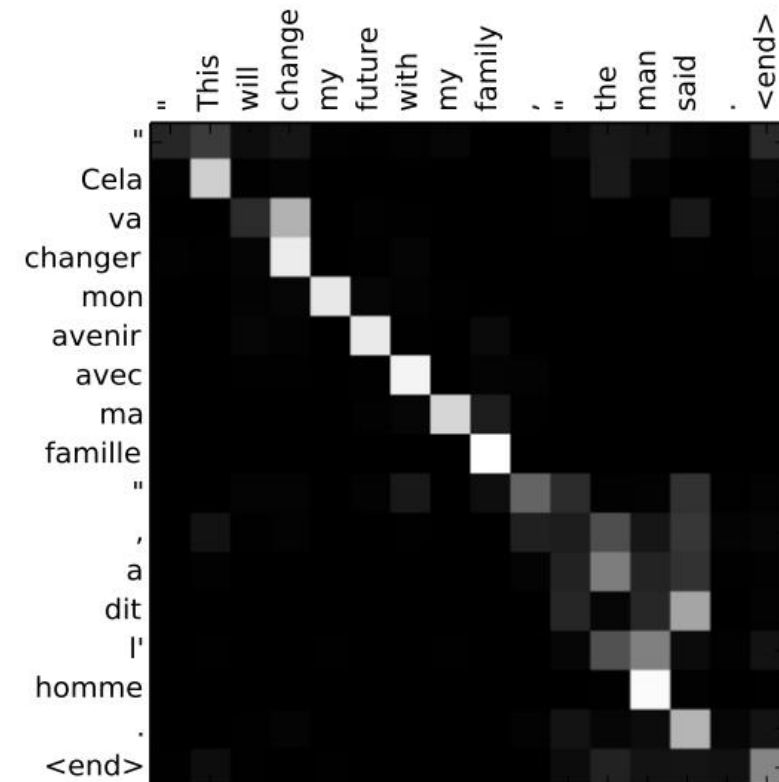
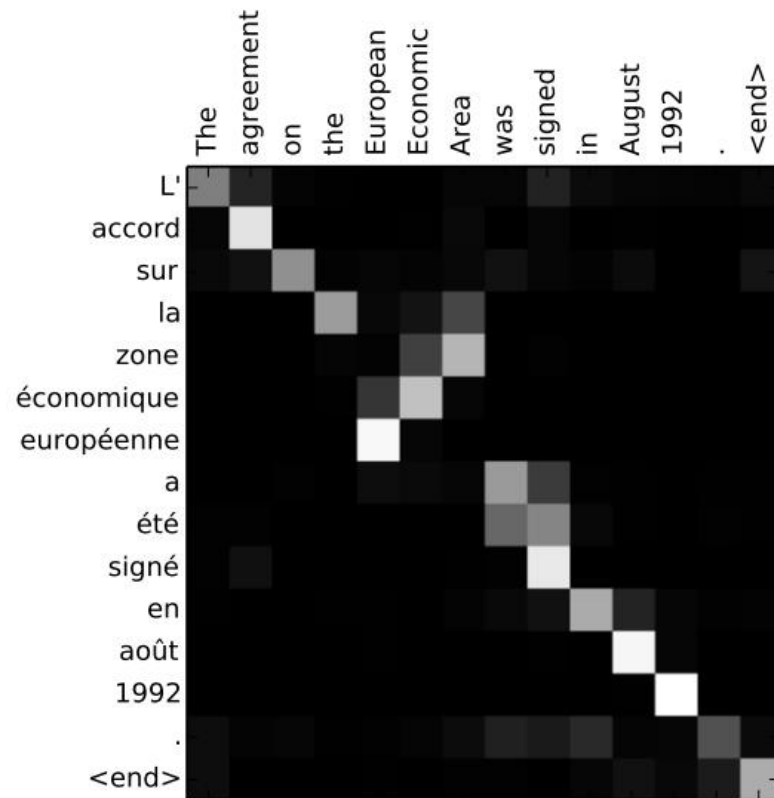
# Attention

When computing the new hidden state  $s_t$ , attention mechanism compute attention scores with all the input tokens (from a bidirectional RNN).



# Attention

An example of machine translation.



From: Attention 2015

# Attention without RNNs

---

It has been proposed to eliminate the RNN component due to:

- The primary function of the RNN is to extract and process sequential features. However, this functionality can be achieved using simpler methods, leading to **reduced computational complexity**.
- RNNs are **not parallelizable**, meaning they cannot efficiently process multiple input sequences simultaneously, which limits their scalability and efficiency in large-scale applications.

# Attention without RNNs

$$x, y \in \mathbb{R}^{d_{model} \times N}$$

$$W_Q, W_K \in \mathbb{R}^{d_k \times d_{model}}$$

$$W_V \in \mathbb{R}^{d_v \times d_{model}}$$

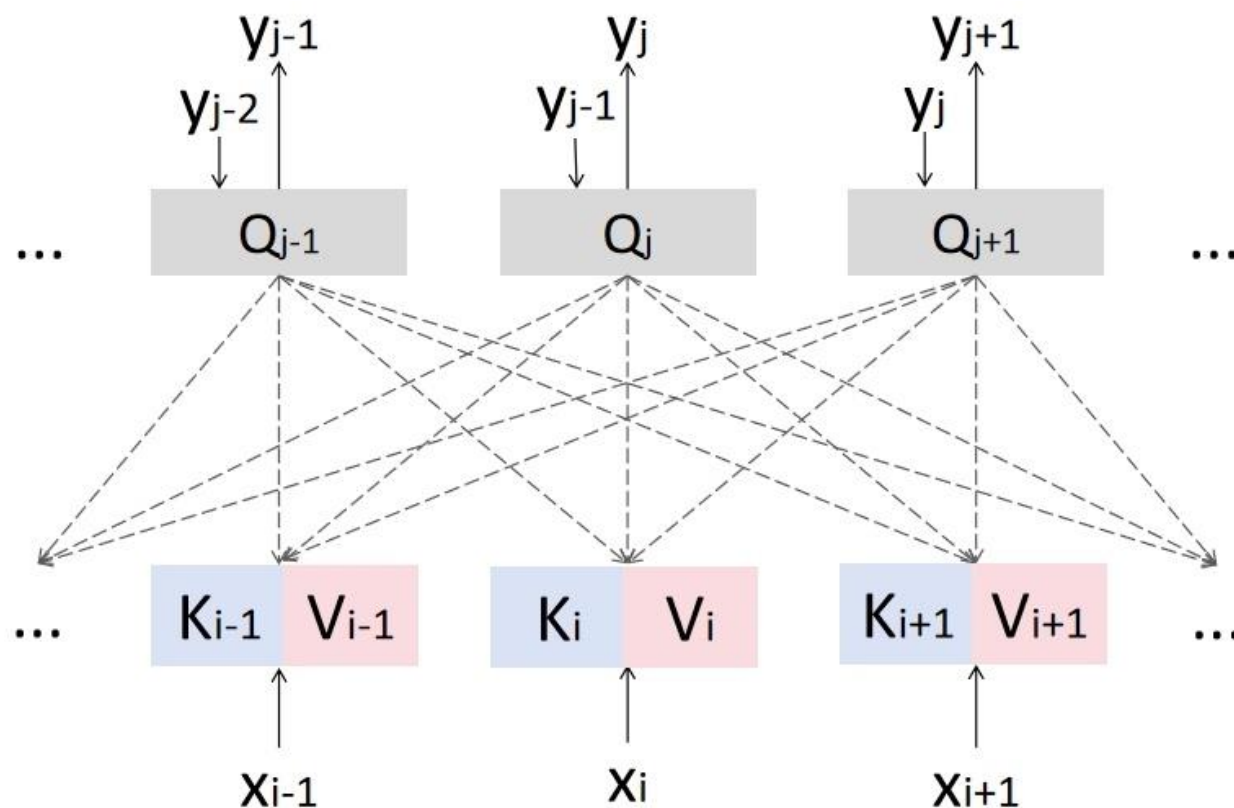
$N$  is text length,  $d_{model}$  is the size of embedding

Learnable weight matrix

$$Q = W_Q \begin{bmatrix} y \\ x \end{bmatrix} \quad \text{output}$$

$$K = W_K \begin{bmatrix} x \\ x \end{bmatrix} \quad \text{input}$$

$$V = W_V \begin{bmatrix} x \\ x \end{bmatrix}$$



Attention score:

$$Q^T K \in \mathbb{R}^{N \times N}$$

$$\alpha_i = \text{Softmax}\left(\frac{Q^T K}{\sqrt{d}}\right)$$

$$y_{output} = \sum_{i=1}^N \alpha_i v_i$$

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

eq in the original paper

# Palimpsest (重寫羊皮紙) 3.0

## 徹底廢黜時間輸送帶：Transformer 的空間革命

從「排隊傳抄」到「全景攤平」，語言模型如何透過 QKV 機制將時間序列轉化為幾何空間。

### Architectural Evolution Matrix

#### 1 初代 RNN (單張羊皮紙)



- 機制：X<sub>t</sub> 暴力塗改覆寫。
- 痛點：舊字成雜訊，導致記憶模糊崩潰。

#### 2 二代 LSTM (密封長卷軸)



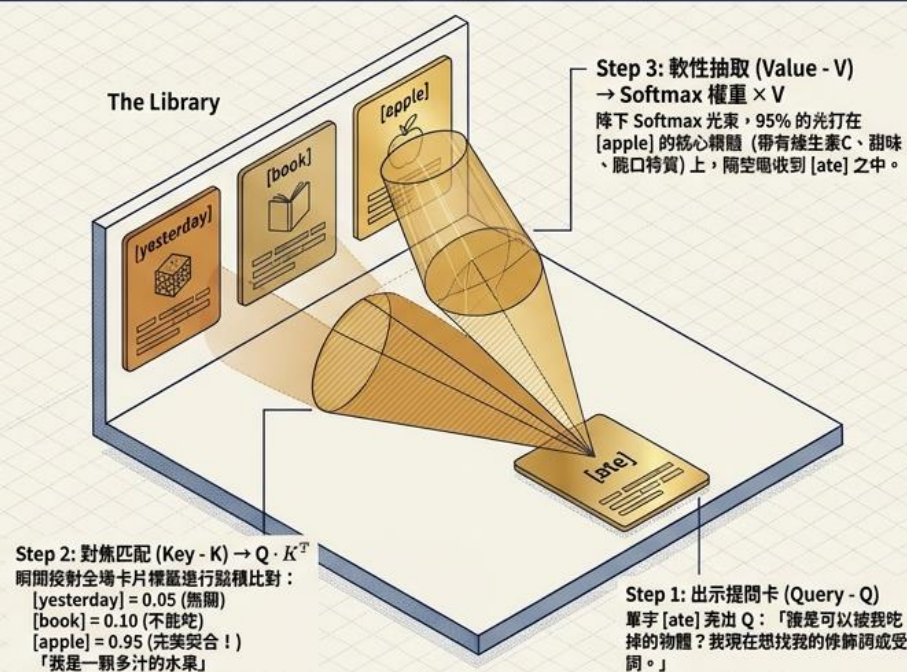
- 機制：主卷軸 C<sub>t</sub> 透過過濾閥門進行局部加法。
- 痛點：序列枷鎖。第 100 頁必須滑過 99 個抄寫員無法平行運算（後面必須等前面）。

#### 3 現代 Transformer (卡片圖書館)



- 機制：全域攤平，每字化為獨立圖書卡片。
- 突破：距離歸零，任意跨步瞬間加權抓取。

### The Core Engine (QKV 三聯檢索卡)



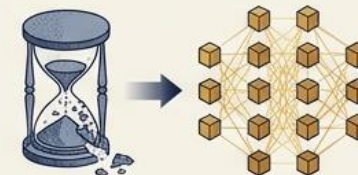
### The Two Century Problems Solved

#### 3A 告別歷史稀釋 (O(1) 恆定記憶)



不管兩張卡片隔了多遠，提問 Q 與標籤 K 皆是一步直達 (O(1) 距離)。第 1 個字與第 1,000 個字對話的清晰度，跟相鄰兩字完全一模一樣。

#### 3B 撕掉「時間束縛」(平行化運算)



$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

雙本 5,000 字的書無需排隊。GPU 在同一個時鐘週期內，透過一次矩陣乘法，瞬間完成全場所卡片兩兩之間的比對。

### BOTTOM FOOTER MODULE

哲學躍遷：處理時間序列，不一定需要「時間的順序」。把時間攤平成幾何空間，賦予每個單字直接去圖書館貨架檢索他人的權力，語言模型才真正走出了記憶衰退的迷宮。

# Why $\sqrt{d}$

---

The model's parameters should undergo normalization, ensuring their average is 0 and variance is 1.

Assume that  $q_i$  and  $k_i$  are random variable with average 0 and variance 1:

$$E(q_i k_i) = E(q_i)E(k_i) = 0$$

$$\begin{aligned} Var(q_i k_i) &= E(q_i^2 k_i^2) - E(q_i k_i)^2 \\ &= E(q_i^2 - 0^2)E(k_i^2 - 0^2) \\ &= E(q_i^2 - E(q_i)^2)E(k_i^2 - E(k_i)^2) \\ &= Var(q_i)Var(k_i) = 1 \end{aligned}$$

# Why $\sqrt{d}$

---

However, after we multiply Q and K, the variance becomes d:

$$\text{Var}(Q^\top K) = \text{Var}\left(\sum_{i=0}^d q_i k_i\right) = d \cdot 1 = d$$

So it should be divided by  $\sqrt{d}$ :

$$\text{Var}\left(\frac{Q^\top K}{\sqrt{d}}\right) = \frac{d}{(\sqrt{d})^2} = 1$$

# Summary

---

Sequential Models:

**RNN:** Recurrent Neural Network, the fundamental NN in NLP tasks.

**LSTM:** A modification of RNNs designed to alleviate gradient issues.

Attentions:

**Attention with RNN:** Avoid gradient vanishing in RNNs.

**Attention without RNN:** Parallelizable and simpler.

# Reference

---

## Stanford NLP:

- <https://web.stanford.edu/~jurafsky/slp3/>

## Attention 2015:

- Bahdanau, D., Cho, K., & Bengio, Y. (2014). Neural machine translation by jointly learning to align and translate. arXiv preprint arXiv:1409.0473.